How AI Agents Actually Work

Step-by-step with Python Code

LLMs • LangChain • Vector DBs • RAG • Memory • Tool Use • Agents

1. What is an LLM?
2. Tokenization
3. Embeddings
4. Vector Databases
5. RAG Pipeline
6. LangChain Setup
7. Prompt Templates
8. Chains
9. Memory
10. Tool Use
11. ReAct Agents
12. LangGraph
13. Production Tips
14. Full Stack Agent

-> Swipe through 14 slides | Full Python code every slide <-

Save this • Comment 'AI' for PDF • Share with your team

Kaushal Singh
Full-Stack Dev | MERN + React Native | Fintech @ Kanmotech Pvt. Ltd.

685,000+ impressions | Follow for AI, Python & System Design
Comment 'AI' for full PDF | Save | Repost to your network

02 What is an LLM?

Large Language Models — the brain of every AI agent

■ What is an LLM?

An LLM (Large Language Model) is a neural network trained on billions of text tokens to predict the next word.

How it works:

- Input: sequence of tokens (text broken into pieces)
- Processing: transformer layers compute attention
- Output: probability distribution over next tokens
- Sampling: pick next token, repeat until done

Key properties:

- Context window: how many tokens it can see at once

GPT-4: 128K | Claude 3: 200K | Gemini: 1M

- Temperature: controls randomness (0=deterministic, 1=creative)
- Parameters: GPT-4 ~1.7T, LLaMA-3 70B, Mistral 7B

Popular models:

- OpenAI: GPT-4o, GPT-4-turbo
- Anthropic: Claude 3.5 Sonnet, Claude 3 Opus
- Meta: LLaMA 3 (open source!)
- Google: Gemini 1.5 Pro, Gemini Flash
- Mistral: 7B, Mixtral 8x7B (open source!)

■ Python: Call an LLM

```python
# pip install openai anthropic

# OpenAI GPT-4
from openai import OpenAI
client = OpenAI(api_key='sk-...')

response = client.chat.completions.create(
    model='gpt-4o',
    messages=[
        {'role': 'system', 'content': 'You are a helpful
        {'role': 'user',   'content': 'Explain RAG in 2
    ],
    temperature=0.7,      # 0=focused, 1=creative
    max_tokens=500,
)
print(response.choices[0].message.content)

# Anthropic Claude
import anthropic
client = anthropic.Anthropic(api_key='sk-ant-...')
msg = client.messages.create(
    model='claude-sonnet-4-20250514',
    max_tokens=1024,
    messages=[{'role': 'user', 'content': 'Hello, Claude
)
print(msg.content[0].text)

# Local LLM via Ollama (free, runs on your machine)
import ollama
res = ollama.chat(model='llama3', messages=[
    {'role': 'user', 'content': 'What is LangChain?'}
])
print(res['message']['content'])
```

03 Tokenization & Embeddings

How text becomes numbers — the foundation of all AI

✂■ Tokenization

Tokenization splits text into tokens — subword pieces.
LLMs don't see letters or words, they see token IDs.

How tokenization works:

- 'Hello world' -> [9906, 1917] (GPT-4 tokens)
- Common words: 1 token each
- Rare words: split into multiple tokens
- 1 token ≈ 4 characters ≈ 0.75 words

Why it matters:

- Context window = max token count (not characters!)
- Cost = tokens in + tokens out
- Some languages use more tokens per word

Embeddings:

An embedding converts text into a vector of numbers.
Similar texts have vectors that are close in space.

- 'cat' and 'kitten' -> nearby vectors
- 'cat' and 'blockchain' -> far apart vectors
- OpenAI embeddings: 1536 dimensions
- Used for: search, similarity, clustering, RAG

■ Tokenize + Embed in Python

```python
# pip install tiktoken openai numpy

# --- TOKENIZATION ---
import tiktoken
enc = tiktoken.encoding_for_model('gpt-4o')

text = 'LangChain makes AI agents easy to build!'
tokens = enc.encode(text)
print(f'Token IDs: {tokens}')
print(f'Count: {len(tokens)} tokens')
# Token IDs: [43, 1901, 8995, 1285, ...]
# Count: 9 tokens

decoded = enc.decode(tokens)
print(f'Decoded: {decoded}')  # original text

# --- EMBEDDINGS ---
from openai import OpenAI
import numpy as np

client = OpenAI(api_key='sk-...')
def embed(text: str) -> list[float]:
    res = client.embeddings.create(
        model='text-embedding-3-small',
        input=text
    )
    return res.data[0].embedding  # 1536 floats

# Cosine similarity: how similar are two texts?
def cosine_sim(a, b):
    a, b = np.array(a), np.array(b)
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg

v1 = embed('Python programming language')
v2 = embed('Coding in Python')
v3 = embed('Football match score')
print(f'Python vs Coding: {cosine_sim(v1, v2):.3f}')  #
print(f'Python vs Football: {cosine_sim(v1, v3):.3f}') #
```

04 Vector Databases

Store, index, and search billions of embeddings at speed

■■ Vector DB Concepts

A vector database stores embeddings and enables extremely fast similarity search (nearest neighbours).

Why vector DBs?

- Regular DB: WHERE name='cat' (exact match)
- Vector DB: find most similar to 'cat' (semantic)
- Returns top-K most similar vectors in milliseconds
- Even across billions of documents!

Popular choices:

- Pinecone — managed, production-ready, easiest
- Chroma — local dev, open source, embedded
- Weaviate — open source, GraphQL API
- Qdrant — Rust-based, very fast, open source
- FAISS — Facebook, in-memory, no persistence
- pgvector — Postgres extension (you know Postgres!)

What gets stored per document:

- id — unique identifier
- embedding — vector of floats [0.12, -0.43, ...]
- metadata — original text, source, date, author

ANN search (Approximate Nearest Neighbour):

- HNSW index: graph-based, fast, high accuracy
- IVF: inverted file, good for billions of vectors

■ Chroma + Pinecone in Python

```python
# pip install chromadb pinecone-client

# --- CHROMADB (local, great for dev) ---
import chromadb

client = chromadb.Client()  # in-memory
# or: chromadb.PersistentClient(path='./chroma_db')

collection = client.create_collection('my_docs')

# Add documents (Chroma auto-embeds with default model)
collection.add(
    documents=[
        'LangChain is a framework for LLM apps',
        'RAG retrieves context before generating',
        'Agents use tools to complete tasks',
    ],
    ids=['doc1', 'doc2', 'doc3']
)

# Semantic search
results = collection.query(
    query_texts=['How do AI agents work?'],
    n_results=2
)
print(results['documents'])  # top 2 matches

# --- PINECONE (managed, production) ---
from pinecone import Pinecone

pc = Pinecone(api_key='pc-...')
index = pc.Index('my-index')

# Upsert vectors
index.upsert(vectors=[
    {'id': 'doc1', 'values': embed('LangChain framework'
     'metadata': {'text': 'LangChain framework', 'source
])

# Query: top 3 similar
res = index.query(vector=embed('AI framework'), top_k=3,
                  include_metadata=True)
for match in res['matches']:
    print(f"Score: {match['score']:.3f} | {match['metada
```

05 RAG Pipeline (Retrieval-Augmented Generation)

Give the LLM your private knowledge — without fine-tuning

RAG Flow: User Query -> Embed -> Search VectorDB -> Inject Context -> LLM -> Answer

■ Why RAG?

LLMs are trained on old data and have no access to your private documents. RAG solves both problems.

Without RAG:

- LLM only knows training data (cutoff date)
- Cannot access your company docs, PDFs, DBs
- Hallucinates facts it doesn't know

With RAG:

- Your documents live in a vector DB
- User question -> find relevant docs
- Inject docs as context -> LLM answers accurately
- Cites sources -> verifiable answers

RAG vs Fine-tuning:

- RAG: dynamic, update docs anytime, cheaper
- Fine-tuning: bakes knowledge into model weights, expensive, static until retrained
- Use RAG first. Fine-tune only if RAG not enough.

Indexing phase (run once):

- Load documents (PDF, web, DB, CSV)
- Split into chunks (512-1024 tokens each)
- Embed each chunk -> store in vector DB

Query phase (every request):

- Embed user query
- Search vector DB -> top-K chunks
- Stuff chunks into prompt -> LLM answers

■ RAG from Scratch in Python

```python
# pip install openai chromadb pypdf

from openai import OpenAI
import chromadb, pypdf, textwrap

client = OpenAI(api_key='sk-...')
chroma = chromadb.Client()
col = chroma.create_collection('docs')

# STEP 1: Load and chunk a PDF
def load_pdf(path):
    reader = pypdf.PdfReader(path)
    text = ''.join(p.extract_text() for p in reader.page
    # Split into 500-char chunks with 50-char overlap
    chunks = textwrap.wrap(text, 500)
    return chunks

# STEP 2: Index documents
def index_docs(path):
    chunks = load_pdf(path)
    col.add(
        documents=chunks,
        ids=[f'chunk_{i}' for i in range(len(chunks))]
    )
    print(f'Indexed {len(chunks)} chunks')

# STEP 3: Query (retrieve + generate)
def rag_query(question: str) -> str:
    # Retrieve top-3 relevant chunks
    results = col.query(
        query_texts=[question], n_results=3
    )
    context = '\n\n'.join(results['documents'][0])

    # Build prompt with retrieved context
    prompt = f'''
    Answer based ONLY on this context:
    {context}

    Question: {question}
    '''

    res = client.chat.completions.create(
        model='gpt-4o',
        messages=[{'role': 'user', 'content': prompt}]
    )
    return res.choices[0].message.content

# Usage
index_docs('my_document.pdf')
answer = rag_query('What is the refund policy?')
print(answer)  # Grounded answer from your doc!
```